

PRIME: Prompt Refinement via Iterative Mutation and Evolution for Robust LLM Annotation

Anonymous ACL submission

Abstract

Large language models are increasingly used as scalable data annotators, but their quality under domain and user distribution shifts remains highly sensitive to prompt design. We introduce **PRIME**, a prompt-only framework that adapts a frozen LLM ensemble by evolving a shared structured prompt through any model exposed via an inference API. PRIME combines an AlphaEvolve-style LLM mutator with an active-learning controller that repeatedly exposes the mutator to hard or high-disagreement examples, consolidates stable rules, and optionally injects synthetic few-shot boundary examples. On the Amazon-Reviews subset of WILDS, PRIME improves the zero-shot ensemble from 50.00% to 55.91% worst-group accuracy and from 71.30% to 75.41% global accuracy on the full test set, outperforming fine-tuned baselines such as LISA. Our ablations show that active batch rotation and synthetic boundary examples are key for robustness. The framework is compatible with black-box models accessed through commercial or self-hosted API endpoints.

1 Introduction

Data annotation is a pivotal stage in developing machine learning systems: model quality critically depends on the representativeness and reliability of training labels. For an extended period, the dominant approach has been manual labeling by domain experts or trained crowdworkers (Snow et al., 2008; Sabou et al., 2014; Klie et al., 2024). Nevertheless, manual annotation suffers from inherent limitations—the human factor, limited throughput, and high cost—that make it difficult to scale to large, evolving corpora.

The rapid advancement of large language models (LLMs) has led to their adoption as scalable annotators (Tan et al., 2024; Gilardi et al., 2023; Wang et al., 2021). LLMs offer high throughput, accessibility, and reduced dependence on per-

example human labor, which is critical for consistent annotation at scale. Despite broad coverage across domains, *domain adaptation* remains essential for high-quality automatic labeling: even strong general-purpose models systematically underperform domain experts on specialized tasks (Tseng et al., 2025; Pangakis et al., 2023). Fine-tuning methods are commonly used (Gururangan et al., 2020; Hu et al., 2022), but they demand substantial compute, engineering effort, and weights access, and are often impractical when models are exposed only through inference APIs.

Another critical factor is the instruction set—the prompt—provided to the model. Recent studies show that LLM outputs are highly sensitive to even minor variations in prompt formulation (Seleznyov et al., 2025; Sclar et al., 2024; Mizrahi et al., 2024), which makes prompt optimization a first-order practical problem rather than a secondary implementation detail. We therefore focus on *prompt-only* domain adaptation: evolving a shared structured prompt that improves ensemble annotation on target-domain data while keeping all annotator weights frozen and compatible with commercial or self-hosted API endpoints.

Prompts are textual instructions that LLMs can both follow and revise, and they have been studied extensively in automatic optimization work (Zhou et al., 2023; Pryzant et al., 2023; Yang et al., 2024). However, many pipelines still rely on human experts to validate generated prompts, on a fixed validation set chosen up front, or on a single best-prompt search that can collapse to local optima. Consequently, prompt engineering often remains dependent on intuition and task-specific expertise rather than on a fully data-driven loop.

We address this gap with **PRIME**, a framework that couples three components: an AlphaEvolve-style LLM mutator for automatic prompt search, a heterogeneous ensemble of frozen LLM annotators for robust majority voting, and an active-learning

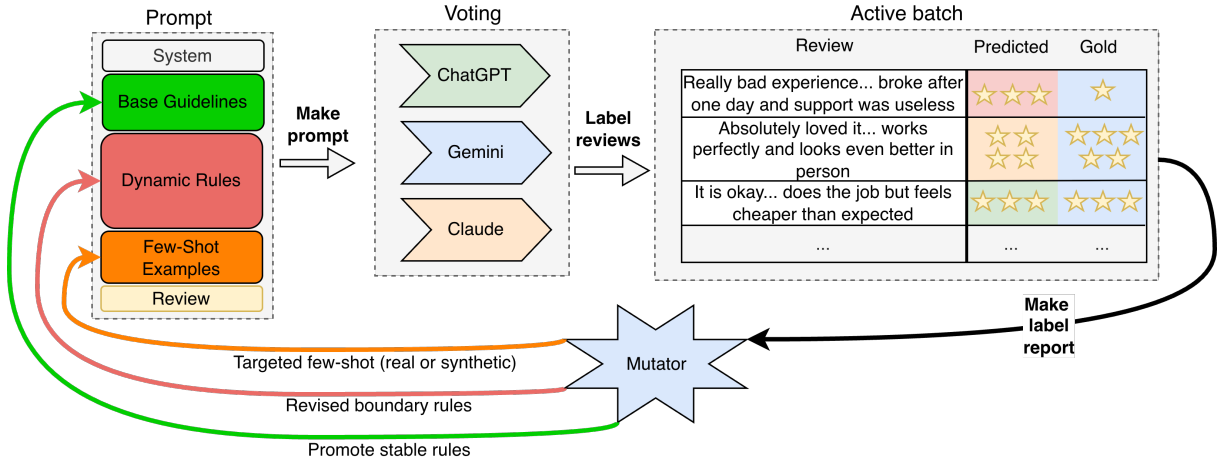


Figure 1: Solution architecture for Prompt Refinement via Iterative Mutation and Evolution. The current structured prompt is instantiated for each review and sent to a LLM ensemble. Majority voting produces predicted labels for the active batch, which are compared against gold labels to form a label report. The mutator then revises the prompt through three update paths: targeted few-shot examples (real or synthetic), revised boundary rules, and promoted stable rules.

controller that repeatedly exposes the mutator to informative errors and disagreement cases (Settles, 2009; Margatina et al., 2023). We selected AlphaEvolve (Novikov et al., 2025) as the inner optimizer because it combines two properties that fit annotation under shift: (i) a genetic algorithm with an LLM mutator, avoiding the need for a differentiable objective; and (ii) a diversity-preserving archive of prompt candidates that mitigates premature convergence.

Conventional prompt tuning on randomly sampled training batches is inefficient and noisy for annotation: many examples are already easy under the current prompt and contribute little signal for the prompt refinement. PRIME instead rebuilds a compact **Hard/Anchor** active batch each cycle from ensemble mistakes and worker disagreement, optionally enriched with synthetic boundary few-shot pairs, so evolutionary updates concentrate on cases that still limit deployment quality. Figure 1 summarizes the loop: ensemble workers label the active batch, a label report is passed to the mutator, and the mutator updates the structured prompt through targeted examples, boundary rules, and stable-rule consolidation.

Our experiments on Amazon WILDS (Koh et al., 2021) show that this prompt-only adaptation achieves competitive performance under out-of-distribution user shifts—a key requirement for deployed annotation systems where test users and product categories differ from the training distribution.

Contributions. This paper makes three main contributions:

1. We couple evolutionary prompt search with active learning for annotation under distribution shift, using an AlphaEvolve-based mutator and dynamic Hard/Anchor batches driven by majority-vote errors and worker disagreement.
2. We introduce a structured prompt representation with separate within-cycle and between-cycle update rules, limiting catastrophic forgetting in prompt space.
3. We demonstrate strong results on Amazon WILDS and ablate the contributions of active learning and synthetic few-shot examples.

2 Related Work

Data annotation with humans and LLMs. Crowd and expert labeling remains the gold standard but suffers from cost, throughput limits, and disagreement; LLMs offer scalable annotation, although validation and domain expertise remain important. Kamen and Kamen (2025) show that heterogeneous LLM voting reduces variance in ensemble prediction. We extend this idea by using ensemble errors and disagreement as the supervision signal for prompt evolution.

Domain adaptation of LLMs. LLM adaptation commonly uses continual pretraining, parameter-efficient fine-tuning such as LoRA, or soft

prompts (Lester et al., 2021; Han et al., 2024). These methods are effective but require weight access and engineering effort; PRIME instead modifies only the input prompt, preserving compatibility with any model reachable through an inference API.

Active learning. Classical active learning selects informative examples by uncertainty, committee disagreement, or diversity, and recent work adapts these ideas to in-context learning (Diao et al., 2024). PRIME couples acquisition with prompt updates: current disagreement determines the next evolutionary batch.

Robustness under distribution shift. WILDS evaluates robustness under real-world shifts, and its Amazon-Reviews split simulates a user shift. Standard baselines such as GroupDRO (Sagawa et al., 2020), IRM (Arjovsky et al., 2019), JTT (Liu et al., 2021), and LISA (Yao et al., 2022) update weights, while naive fine-tuning can harm OOD accuracy (Kumar et al., 2022). We evaluate whether LLM annotators can be adapted purely through prompts.

Automatic prompt optimization and evolutionary search. Prompt sensitivity has motivated automatic prompt optimization, including APE, APO, OPRO, DSPy (Khatab et al., 2024), PromptBreeder (Fernando et al., 2024), and Evo-Prompt (Guo et al., 2024). A parallel line uses LLMs as mutators in program search: Fun-Search (Romera-Paredes et al., 2024) combines LLM mutation with large-scale evaluation and diversity-preserving archives. PRIME applies this paradigm to data annotation under distribution shift, replacing fixed-subset optimization with active acquisition and structured prompt updates.

3 Method

3.1 Framework Overview

We propose **Prompt Refinement via Iterative Mutation and Evolution (PRIME)**, a framework that adapts an ensemble of frozen LLMs to a specific domain solely by evolving their shared instruction set (the prompt). The framework operates on two nested levels. The inner loop uses OpenEvolve¹ (Sharma, 2025), an open-source implementation of the AlphaEvolve paradigm, where

¹<https://github.com/algorithmicsuperintelligence/openevolve>

an LLM acts as a mutator to iteratively refine prompt candidates against an active batch of training data. The outer loop implements an active-learning strategy: it evaluates the current best prompts, consolidates their rules, and dynamically updates the active batch by sampling the most informative examples from an unlabeled pool. Figure 1 summarizes the architecture.

3.2 Problem Formulation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ denote a labeled dataset of reviews (or, more generally, texts) with ordinal labels $y_i \in \{1, \dots, 5\}$. We assume access to a fixed ensemble of M annotator LLMs $\{f_m\}_{m=1}^M$, each invoked with a shared natural-language prompt P . The ensemble prediction is obtained by majority vote:

$$\hat{y}(x; P) = \text{mode}(\{f_m(x; P)\}_{m=1}^M). \quad (1)$$

A prompt P is represented as a structured document (Section 3.3) rather than a monolithic string. Our goal is to find a prompt that maximizes annotation quality under distribution shift while keeping all f_m frozen:

$$P^* \in \arg \max_{P \in \mathcal{P}} \mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{test}}} [\mathbf{1}\{\hat{y}(x; P) = y\}], \quad (2)$$

where $\mathcal{P}$ is the space of valid structured prompts and $\mathcal{D}_{\text{test}}$ follows an out-of-distribution (OOD) user split (Section 4). Because $\mathcal{P}$ is discrete and non-differentiable, we optimize P with a genetic search guided by a scalar fitness $\mathcal{F}(P)$ (Section 3.6) and an active-learning controller that selects which examples enter each evolutionary episode.

3.3 Structured Prompt Representation

Monolithic prompts are fragile under LLM-driven mutation: fixing a new error often deletes a rule that previously worked—a form of *catastrophic forgetting in prompt space*. We therefore decompose P into four interpretable sections:

<System> Role definition and a concise task description for the annotator ensemble.

<BaseGuidelines> Stable rules promoted from successful evolutionary runs. Updated only during *consolidation* (Section 3.7).

<DynamicRules> Experimental hypotheses refined by the mutator during the inner loop. This is the primary search space within each active-learning cycle.

<FewShotExamples> Few-shot demonstrations, refreshed to cover current confusion patterns and boundary cases.

Two-level update policy.

Within-cycle (Level 1): during each outer AL cycle, the mutator may edit <DynamicRules> and <FewShotExamples> only (Algorithm 1, Level 1).

Between-cycle (Level 2): after a cycle ends, consolidation may merge proven rules into <BaseGuidelines> (Section 3.7).

Level 1 enables fast adaptation to the current active batch; Level 2 promotes only durable rules into the long-term prompt backbone. This separation preserves accumulated domain knowledge while still allowing rapid adaptation to newly acquired hard examples.

3.4 Synthetic Few-Shot Generator

When enabled, an auxiliary LLM call runs at the beginning of each outer cycle, *before* Level 1 evolution (Algorithm 1, optional step), to provide controlled minimal-pair contrasts for dominant error boundaries. The generator reads top confusion pairs (e.g., gold rating 3 $\rightarrow$ predicted rating 4), misclassified examples, and current <DynamicRules>, then creates synthetic *twin* review pairs that differ in one rating-relevant cue while covering the 1–5 scale. The resulting examples are either injected as a mutator hint or used to refresh <FewShotExamples>. Main experiments keep this module enabled; Section E ablates it, and Appendix F shows a concrete example set. The motivation is that many Amazon errors are adjacent-class errors rather than arbitrary label flips: reviews with mixed praise and criticism are often confused between 3 and 4 stars, while severe product failures may be confused between 1 and 2 stars. Synthetic twin pairs make these local boundaries explicit by holding most review content fixed and changing only one rating-relevant cue. This gives the mutator a compact description of what should change the label without adding noisy real examples to the active batch.

3.5 Active Learning Loop and Batch Construction

Evaluating every prompt candidate on the full training pool at each mutation is prohibitively expensive. Instead, we maintain a dynamic **active batch** $\mathcal{B}_t$ of

size $|\mathcal{B}_t| = B$ (e.g., $B=80$), rebuilt at the start of each outer cycle t . Prompt discovery starts from a compact but diverse seed training pool sampled via clustering from the full WILDS training split; during active learning, this pool can be expanded with additional examples from the full training split whenever the controller needs new hard cases. The validation and development test splits used for cycle-level monitoring are constructed representatively across product categories (Appendix D).

Seen / Unseen pool. The training pool is partitioned into **seen** examples $\mathcal{S}_t$ already evaluated at least once and **unseen** examples $\mathcal{U}_t$ not yet used for adaptation; initially, $\mathcal{S}_0 = \emptyset$ and $\mathcal{U}_0 = \{1, \dots, N\}$.

Hard / Anchor split. After evaluation with the current prompt, each seen example $i \in \mathcal{S}_t$ is labeled using ensemble outputs $\{f_m(x_i; P)\}_{m=1}^M$:

$$d_i = \frac{1}{\binom{M}{2}} \sum_{1 \leq a < b \leq M} |f_a(x_i; P) - f_b(x_i; P)|, \quad (3)$$

where $y_{\min}=1$ and $y_{\max}=5$, so d_i is normalized to $[0, 1]$ over the rating scale. Example i is assigned to the **Hard** set $\mathcal{H}_t$ if the ensemble majority vote is incorrect ($\hat{y}(x_i; P) \neq y_i$) or worker disagreement exceeds a threshold ($d_i > \tau$), and to the **Anchor** set $\mathcal{A}_t$ otherwise. In our reported runs we use $\tau=0$ (Appendix A), meaning that any worker-level disagreement is treated as uncertain even if the majority vote is correct. Intuitively, $\mathcal{H}_t$ contains examples the ensemble majority misclassifies or examples on which workers disagree; $\mathcal{A}_t$ contains examples the current prompt handles correctly and consistently.

Batch assembly. The active batch is assembled with a fixed quota of Hard and Anchor examples (e.g., 70% Hard and 30% Anchor, with $\rho=0.7$). We use $B=80$ and $\rho=0.7$ in all reported runs: enough Hard examples for diverse error coverage, while Anchor examples preserve already mastered behavior. To avoid overfitting to a narrow cluster of errors, Hard examples are selected via k -means clustering on sentence embeddings (Margatina et al., 2021), taking diverse representatives from each cluster; Anchor examples are drawn with stratified sampling over rating labels. Appendix A reports the embedding model and other run-level hyperparameters.

Pool expansion. As the prompt improves within a cycle, previously hard examples are reclassified

Algorithm 1 PRIME — one outer cycle

Require: Seed prompt P_t ; active batch size B ; unseen pool $\mathcal{U}_t$

Ensure: Updated prompt P_{t+1}

- 1: Build active batch $\mathcal{B}_t$ from hard/anchor pools $\mathcal{H}_t, \mathcal{A}_t$ $\triangleright$ Hard/Anchor quota

Optional: refresh `<FewShotExamples>` via synthetic twin-pair generator

Level 1 (evolution)

- 2: Run AlphaEvolve on $\mathcal{B}_t$ with fitness $\mathcal{F}(P)$ $\triangleright$ Eq. (4)

- 3: Re-evaluate $\mathcal{B}_t$; update $\mathcal{H}_t$ and $\mathcal{A}_t$

- 4: Validate on holdout; compute $\mathcal{S}(P)$ $\triangleright$ Eq. (6)

Level 2 (consolidation)

- 5: Merge top- K archive prompts into P_t^{cons}

- 6: **if** $\mathcal{S}(P_t^{\text{cons}}) \geq \mathcal{S}(P_t) - \delta$ **then**

- 7: $P_{t+1} \leftarrow P_t^{\text{cons}}$

- 8: **else**

- 9: $P_{t+1} \leftarrow$ best Level-1 evolution prompt

- 10: **end if**

- 11: **if** $|\mathcal{H}_t| \leq \theta_{\text{exp}}$ **then**

- 12: Expand $\mathcal{S}_t$ from $\mathcal{U}_t$ (farthest-from-anchor sampling)

- 13: **end if**

- 14: **return** P_{t+1}
-

as anchors, so $|\mathcal{H}_t|$ typically shrinks even though no examples leave the dataset. When the hard pool falls below a threshold θ_{exp} (e.g., 5–10 examples), the controller *expands* the seen pool by importing new examples from the unseen pool $\mathcal{U}_t$. Candidates are ranked by semantic distance to the current anchor set (cosine distance in embedding space) so that newly added examples are maximally dissimilar from already mastered cases—encouraging coverage of new failure modes rather than redundant near-duplicates.

3.6 Evolutionary Objective

The inner loop follows the AlphaEvolve paradigm: an LLM acts as a mutator in a genetic algorithm, maintaining a diverse archive of prompt candidates (MAP-Elites-style database) and selecting parents for crossover/mutation without gradient access.

Fitness on the active batch. For each candidate prompt P , fitness is a weighted combination of batch accuracy and ensemble agreement, minus a

length penalty:

$$\mathcal{F}(P) = w_1 \cdot \text{Acc}_{\text{hard}}(P) + w_2 \cdot \text{Acc}_{\text{anchor}}(P) + w_3 \cdot \kappa_{\text{hard}}(P) - \mathcal{P}_{\text{len}}(P), \quad (4)$$

where:

- $\text{Acc}_{\text{hard}}(P)$ and $\text{Acc}_{\text{anchor}}(P)$ are majority-vote accuracies on $\mathcal{B}_t^{\text{hard}}$ and $\mathcal{B}_t^{\text{anchor}}$;
- $\kappa_{\text{hard}}(P)$ is the mean pairwise Cohen’s κ among ensemble members on Hard examples, rewarding prompts that increase annotator consensus;
- $\mathcal{P}_{\text{len}}(P)$ penalizes prompts longer than L_{max} tokens (e.g., $L_{\text{max}}=8000$) to discourage prompt bloat.

In our implementation we use $(w_1, w_2, w_3) = (0.5, 0.3, 0.2)$ (see Section 5.2). The mutator also receives **error artifacts**: misclassified reviews, top confusion pairs (e.g., gold 3 $\rightarrow$ pred 4), and anchor-regression cases formatted as natural-language feedback.

Combined validation score. For full validation and test evaluation (not the inner-loop fitness), we use a unified combined score that balances global accuracy, worst-group robustness, ordinal error severity, and ensemble agreement:

$$C_{\text{val}}(P) = 0.4 R_{\text{global}}^{\text{val}} + 0.3 R_{\text{worst}}^{\text{val}} + 0.3 \left(1 - \frac{\text{MAE}^{\text{val}}}{4} \right) + 0.1 \max(0, \bar{\kappa}^{\text{val}}), \quad (5)$$

clipped to $[0, 1]$, where $\bar{\kappa}^{\text{val}}$ is the mean quadratic-weighted Cohen’s κ between ensemble workers on the validation set. This is *not* the same objective as Eq. (4): C_{val} is computed on the full validation split after each outer cycle, whereas $\mathcal{F}(P)$ is evaluated only on the small active batch during evolution.

Outer-cycle selection score. Prompts retained across cycles are ranked on the validation split using a smoothed score that rewards robustness under user shift:

$$\mathcal{S}(P) = \overline{C}_{\text{val}}^{(3)}(P) + \lambda_w \cdot \overline{R}_{\text{worst}}^{(3)}(P) - \lambda_\sigma \cdot \sigma(C_{\text{val}}^{(3)}(P)), \quad (6)$$

Method	R_{worst} ($\uparrow$)	R_{global} ($\uparrow$)	MAE ($\downarrow$)	Mean κ ($\uparrow$)
<i>Fine-tuned baselines (from WILDS leaderboard)</i>				
ERM (Koh et al., 2021)	53.80%	-	-	-
GroupDRO (Sagawa et al., 2020)	53.30%	-	-	-
LISA (Yao et al., 2022)	54.70%	71.30%	-	-
<i>Frozen LLM baselines (Zero-shot)</i>				
Single LLM (gpt-4o-mini)	53.18%	71.86%	0.3019	-
Baseline Ensemble (Initial Prompt)	50.00%	71.30%	0.3076	0.6973
<i>Ours (Prompt-only adaptation)</i>				
PRIME	55.91%	75.41%	0.2562	0.8815

Table 1: **Main results on the Amazon WILDS full test set.** R_{worst} is the 10th percentile user accuracy (primary robustness metric). PRIME outperforms both the baseline ensemble and state-of-the-art fine-tuning methods (LISA, GroupDRO).

where $\bar{\cdot}^{(3)}$ denotes the trailing three-cycle average of per-cycle values, $\sigma(\cdot)$ is the standard deviation over the same window, and we use $\lambda_w=0.15$, $\lambda_\sigma=0.30$. The best prompt by $\mathcal{S}(P)$ is stored as `best_val_prompt` for final holdout evaluation. Fitness weights $(w_1, w_2, w_3)=(0.5, 0.3, 0.2)$ were chosen to prioritize Hard-batch accuracy, where most evolutionary gains originate; early Home & Kitchen i.i.d. evaluation pilots showed qualitatively similar rankings under moderate perturbations of B , ρ , and (w_1, w_2, w_3) , though we did not run an exhaustive grid search (see Section 5.2).

3.7 Consolidation and Full Algorithm

Between outer cycles, we run a **consolidation** step (Level 2). A separate LLM inspects the top- K prompts from the evolutionary archive together with validation metrics and error artifacts. It merges near-duplicate rules, tightens wording, and promotes robust heuristics from `<DynamicRules>` into `<BaseGuidelines>`. A consolidated prompt is accepted only if its validation score does not regress beyond small tolerances compared to the cycle-best evolution prompt; otherwise the evolution winner is kept.

Algorithm 1 summarizes one outer cycle. Figure 1 illustrates how the label report closes the feedback loop between active-batch evaluation and structured-prompt updates. Appendix G provides a more detailed schematic of how cycle-level rule edits are consolidated into the structured prompt over multiple active-learning rounds.

4 Experiments

4.1 Experimental Setup

Dataset. We evaluate our framework on the Amazon-Reviews subset of the WILDS benchmark. The task is 5-way sentiment classification (1 to 5 stars) based on review text. Crucially, WILDS enforces a strict out-of-distribution (OOD) evaluation protocol: the dataset is partitioned by user ID, ensuring that reviewers in the validation and test sets are disjoint from those in the training set. This simulates a realistic deployment scenario where an annotation system must generalize to the idiosyncrasies of unseen annotators. This is relevant for prompt-based annotation because new users may differ in verbosity, sarcasm, and rating calibration even when the label space remains unchanged. After prompt selection, the resulting prompt is evaluated once on the full, uncapped WILDS test set for the main results in Table 1. Appendix D reports category-level statistics for the prompt-discovery splits.

Ensemble Configuration. To mitigate the high variance inherent to single-LLM zero-shot classification, we employ a heterogeneous ensemble of three frozen, API-accessible models: gpt-4o-mini, gemini-2.5-flash, and claude-3.5-haiku. The ensemble’s final prediction is determined by majority vote. All models receive the exact same structured prompt evolved by our framework. The mutator LLM driving AlphaEvolve is gemini-2.5-pro. The main prompt-discovery configuration used to obtain the final prompt is summarized in Appendix A.

Fine-tuned baselines. ERM is the standard WILDS DistilBERT-base-uncased classifier trained

with cross-entropy on the 5-way star label and selected by OOD validation performance. GroupDRO uses the same backbone and training hyperparameters, but optimizes a group-robust objective over reviewer/domain groups to reduce worst-group loss. LISA is a DistilBERT-based selective manifold-mixup method that interpolates same-label examples across domains to learn invariant predictors.

Metrics. Following the standard WILDS protocol, our primary metric is worst-group accuracy (R_{worst}), defined as the 10th percentile of accuracy across all users in the test set. This metric heavily penalizes models that overfit to majority sub-populations while failing on minority groups. We also report global accuracy (R_{global}) over all test examples, Mean Absolute Error (MAE) to capture the severity of ordinal misclassifications, and mean pairwise Cohen’s κ to measure inter-annotator agreement within the ensemble.

4.2 Main Results

Table 1 presents the performance of PRIME against both fine-tuning baselines and frozen-LLM approaches on the full, uncapped WILDS test set.

Comparison with LLM Baselines. The baseline ensemble, equipped with a standard zero-shot instruction, achieves an R_{worst} of only 50.00%, underperforming even a single gpt-4o-mini model (53.18%). This highlights a known failure mode of ensembling: if the prompt is ambiguous, heterogeneous models will fail in diverse ways, destroying the consensus required for a reliable majority vote (evidenced by a low κ of 0.6973). PRIME, by evolving the prompt to clarify these ambiguities, boosts R_{worst} by an absolute 5.91% and raises ensemble agreement κ to 0.8815. Furthermore, the MAE drops significantly from 0.3076 to 0.2562, indicating that the evolved prompt effectively eliminates severe misclassifications (e.g., confusing 1 star with 5 stars).

Comparison with fine-tuning Methods. Remarkably, our prompt-only adaptation outperforms LISA (54.70%), a state-of-the-art method that requires computationally expensive fine-tuning of the model’s weights. Appendix C summarizes the public ERM/GroupDRO/LISA baseline configurations used for this comparison. This demonstrates that for complex reasoning tasks like sentiment annotation, refining the task definition (the prompt) can

be a more effective domain adaptation strategy than updating the model’s internal representations.

5 Analysis and Ablation

Unless noted otherwise, ablations use the same ensemble and evaluation protocol as Section 4, but may differ in train-pool size or enabled optional modules. To understand the contribution of individual framework components, we conduct a series of ablation studies and analyze the dynamics of the active learning loop.

5.1 Evolution Dynamics and the Impact of Active Learning

We compare full PRIME (active learning with synthetic few-shot enabled) against a *Plain Evolution* baseline during prompt discovery. In Plain Evolution, AlphaEvolve optimizes the prompt on a fixed, randomly sampled subset of the training pool (matched in size to our active batch), without Hard/Anchor partitioning, pool expansion, or synthetic few-shot injection.

Figure 2 plots four metrics on the representative development test split per outer AL cycle (Base, cycles 0–5, and Final). Plain Evolution improves only marginally after the first cycle: R_{global} rises from ~ 0.76 to ~ 0.78 and then flattens, while R_{worst} jumps once at cycle 1 ($\sim 0.53 \rightarrow \sim 0.59$) and remains nearly flat thereafter. PRIME reaches substantially higher ceilings on every metric—for example, R_{worst} peaks at ~ 0.67 (vs. ~ 0.59 for Plain Evolution) and MAE drops to ~ 0.18 at cycle 5 (vs. ~ 0.23 – 0.24).

PRIME is not monotonic: all four panels show a synchronized regression at cycle 4 (e.g., R_{worst} falls from ~ 0.67 to ~ 0.56), consistent with a difficult pool-expansion or batch-refresh event, followed by recovery at cycle 5 and a strong Final checkpoint.

The combined score, computed with the same weighting as Eq. (5), mirrors this pattern and peaks at ~ 0.91 for PRIME versus ~ 0.865 for Plain Evolution. Overall, active batch rotation drives higher variance but unlocks a higher robustness ceiling than static-batch evolution.

5.2 Hyperparameter Sensitivity

Our main all-categories configuration fixes $B=80$, $\rho=0.7$, fitness weights (0.5, 0.3, 0.2), and selection weights $(\lambda_w, \lambda_\sigma)=(0.15, 0.30)$ after pilot runs on a Home & Kitchen subsample. We did not

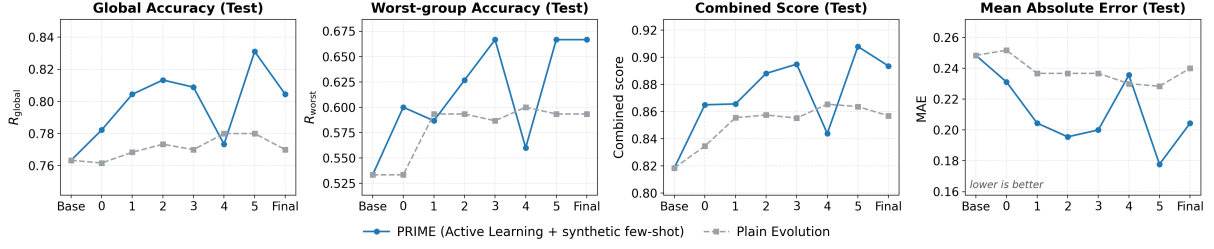


Figure 2: **Metric evolution per AL cycle (development test split).** Four metrics tracked from the Base prompt through outer cycles 0–5 and the Final checkpoint during prompt discovery. **Blue (solid):** PRIME with active learning and synthetic few-shot. **Grey (dashed):** Plain Evolution on a fixed training subset. PRIME reaches higher peak and final values on R_{global} , R_{worst} , and combined score, and achieves lower MAE. Plain Evolution remains comparatively flat after its early gains, while PRIME shows a temporary regression at cycle 4 followed by strong recovery.

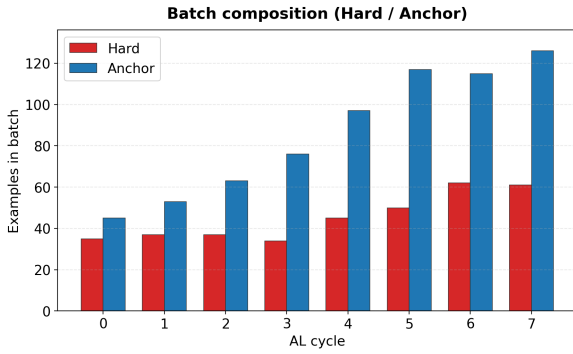


Figure 3: **Active Pool Dynamics.** As the prompt evolves, examples migrate from the Hard pool to the Anchor pool. Periodic spikes indicate pool expansion events where new challenging examples are imported from the Unseen pool.

perform an exhaustive grid search over all knobs. Informal checks varying $B \in \{60, 80, 100\}$ and $\rho \in \{0.6, 0.7, 0.8\}$ preserved the qualitative ranking (PRIME > plain evolution) and similar validation curves, which motivated locking the default values before the final full-test evaluation. Because evolutionary search is stochastic, exact scores can shift by roughly 0.5–1.0 percentage points on R_{worst} under API noise; we therefore report full-test numbers from a single locked configuration rather than tuning hyperparameters on the test set.

Finally, Appendix E reports the synthetic few-shot ablation. Removing the generator reduces Hard-pool accuracy from 38.0% to 26.8%, supporting its role in clarifying adjacent rating boundaries.

5.3 Active Pool Dynamics

The success of PRIME relies on the continuous migration of examples from the Hard pool to the Anchor pool.

Figure 3 illustrates this dynamic across active

learning cycles. As prompts resolve current failures, Hard examples move to Anchor; when Hard is depleted below θ_{exp} , pool expansion imports semantically distant unseen examples, producing controlled spike-and-recovery events. These spikes indicate deliberate exposure to new data regions rather than optimization failure, while recovery shows that PRIME is improving coverage rather than memorizing one static batch.

6 Conclusion

In this work, we introduced PRIME, a framework that adapts an ensemble of large language models to a target annotation domain entirely through prompt optimization. By coupling AlphaEvolve-style mutation with dynamic Hard/Anchor data pools, PRIME keeps prompt search focused on unresolved edge cases, reducing premature convergence and catastrophic forgetting in prompt space.

On Amazon WILDS, PRIME improves upon the zero-shot baseline ensemble, increasing worst-group accuracy (R_{worst}) by nearly 6% absolute, and outperforms state-of-the-art fine-tuning methods such as LISA. Ablation studies confirmed that both the active-learning loop and the injection of synthetic boundary examples are critical for refining the prompt’s decision boundaries and resolving the hardest edge cases. These gains suggest that much of the remaining annotation error comes from underspecified task instructions rather than from missing model capacity alone. Overall, rigorous, data-driven instruction tuning provides a scalable alternative to traditional fine-tuning for complex annotation tasks, especially when models are accessed only through inference APIs.

Limitations

While PRIME provides a highly effective prompt-only mechanism for domain adaptation, it introduces specific trade-offs that practitioners should consider.

API Cost vs. Compute Cost. PRIME shifts the burden of domain adaptation from GPU compute (required by fine-tuning methods such as LISA or LoRA) to API token usage. The evolutionary search, combined with active batch evaluation and consolidation, requires tens of thousands of API calls to the mutator and ensemble models. Although our active-learning controller significantly reduces this cost compared to evaluating on the full training set, the prompt discovery phase remains more expensive in terms of API billing than a simple zero-shot approach. Appendix B provides a token-level cost estimate for the main run.

Inference Token Overhead. The evolved structured prompt, which includes detailed `<DynamicRules>` and `<FewShotExamples>`, is inherently longer than a monolithic zero-shot instruction. Despite our fitness penalty for prompt length (P_{len}), applying the final evolved prompt to millions of unlabeled examples in a production setting will incur higher per-example inference costs due to the increased input token count.

Dependence on Pretrained Knowledge. Because the ensemble remains frozen, PRIME’s success is bounded by the latent capabilities of the underlying LLMs. For tasks requiring highly specialized, proprietary, or rapidly changing factual knowledge not present in the models’ pretraining data, prompt optimization alone cannot inject new facts. In such scenarios, traditional fine-tuning or Retrieval-Augmented Generation (RAG) remains necessary.

References

Martín Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. 2019. [Invariant risk minimization](#). *arXiv preprint arXiv:1907.02893*.

Shizhe Diao, Pengcheng Wang, Yong Lin, Rui Pan, Xiang Liu, and Tong Zhang. 2024. [Active prompting with chain-of-thought for large language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1330–1350. ArXiv:2302.12246.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. [Promptbreeder: Self-referential self-improvement via prompt evolution](#). In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. ArXiv:2309.16797.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. [ChatGPT outperforms crowd workers for text-annotation tasks](#). *Proceedings of the National Academy of Sciences (PNAS)*, 120(30):e2305016120.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. [Connecting large language models with evolutionary algorithms yields powerful prompt optimizers](#). In *International Conference on Learning Representations (ICLR)*. ArXiv:2309.08532.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. 2020. [Don’t stop pretraining: Adapt language models to domains and tasks](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8342–8360. ArXiv:2004.10964.

Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. [Parameter-efficient fine-tuning for large models: A comprehensive survey](#). *Transactions on Machine Learning Research (TMLR)*. ArXiv:2403.14608.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations (ICLR)*. ArXiv:2106.09685.

Ariel Kamen and Yakov Kamen. 2025. [Majority rules: LLM ensemble is a winning approach for content categorization](#). *arXiv preprint arXiv:2511.15714*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. [DSPy: Compiling declarative language model calls into state-of-the-art pipelines](#). In *International Conference on Learning Representations (ICLR)*. ArXiv:2310.03714.

Jan-Christoph Klie, Richard Eckart de Castilho, and Iryna Gurevych. 2024. [Analyzing dataset annotation quality management in the wild](#). *Computational Linguistics*, 50(3):817–866.

Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, Tony Lee, Etienne David, Ian Stavness, Wei Guo, Berton A. Earnshaw, Imran S. Haque, Sara Beery, Jure Leskovec, Anshul Kundaje, and 5 others. 2021. [WILDS: A benchmark of in-the-wild distribution shifts](#). In *Proceedings of*

701	<i>the 38th International Conference on Machine Learning (ICML)</i> , pages 5637–5664. ArXiv:2012.07421.	757
702		758
703	Ananya Kumar, Aditi Raghunathan, Robbie Jones,	759
704	Tengyu Ma, and Percy Liang. 2022. Fine-tuning can	760
705	distort pretrained features and underperform out-of-	761
706	distribution . In <i>International Conference on Learn-</i>	762
707	<i>ing Representations (ICLR)</i> . ArXiv:2202.10054.	763
708	Brian Lester, Rami Al-Rfou, and Noah Constant. 2021.	764
709	The power of scale for parameter-efficient prompt	765
710	tuning . In <i>Proceedings of the 2021 Conference on</i>	766
711	<i>Empirical Methods in Natural Language Processing</i>	767
712	<i>(EMNLP)</i> , pages 3045–3059. ArXiv:2104.08691.	768
713		769
714	Evan Z. Liu, Behzad Haghgoo, Annie S. Chen, Aditi	770
715	Raghunathan, Pang Wei Koh, Shiori Sagawa, Percy	771
716	Liang, and Chelsea Finn. 2021. Just train twice:	772
717	Improving group robustness without training group	773
718	information . In <i>Proceedings of the 38th International</i>	774
719	<i>Conference on Machine Learning (ICML)</i> , pages	775
720	6781–6792. ArXiv:2107.09044.	
721	Katerina Margatina, Timo Schick, Nikolaos Aletras,	776
722	and Jane Dwivedi-Yu. 2023. Active learning prin-	777
723	ciples for in-context learning with large language	778
724	models . In <i>Findings of the Association for Computa-</i>	779
725	<i>tional Linguistics: EMNLP 2023</i> , pages 5011–5034.	780
	ArXiv:2305.14264.	781
726		
727	Katerina Margatina, Giorgos Vernikos, Loïc Barrault,	782
728	and Nikolaos Aletras. 2021. Active learning by ac-	783
729	quiring contrastive examples . In <i>Proceedings of the</i>	784
730	<i>2021 Conference on Empirical Methods in Natural</i>	785
731	<i>Language Processing</i> , pages 650–663, Online and	786
732	Punta Cana, Dominican Republic. Association for	787
	Computational Linguistics.	788
733		789
734	Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror,	790
735	Dafna Shahaf, and Gabriel Stanovsky. 2024. State	791
736	of what art? a call for multi-prompt LLM evaluation .	792
737	<i>Transactions of the Association for Computational</i>	
	<i>Linguistics (TACL)</i> , 12:933–949. ArXiv:2401.00595.	
738	Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Em-	793
739	ilien Dupont, Po-Sen Huang, Adam Zsolt Wagner,	794
740	Sergei Shirobokov, Borislav Kozlovskii, Francisco	795
741	J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abi-	796
742	gail See, Swarat Chaudhuri, George Holland, Alex	797
743	Davies, Sebastian Nowozin, Pushmeet Kohli, and	798
744	Matej Balog. 2025. AlphaEvolve: A coding agent for	799
745	scientific and algorithmic discovery . <i>arXiv preprint</i>	800
746	<i>arXiv:2506.13131</i> .	
747	Nicholas Pangakis, Samuel Wolken, and Neil Fasching.	801
748	2023. Automated annotation with generative AI re-	802
749	quires validation . <i>arXiv preprint arXiv:2306.00176</i> .	803
750		804
751	Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chen-	805
752	guang Zhu, and Michael Zeng. 2023. Automatic	806
753	prompt optimization with “gradient descent” and	807
754	beam search . In <i>Proceedings of the 2023 Con-</i>	
755	<i>ference on Empirical Methods in Natural Lan-</i>	
756	<i>guage Processing (EMNLP)</i> , pages 7957–7968.	
	ArXiv:2305.03495.	
	Bernardino Romera-Paredes, Mohammadamin	808
	Barekatin, Alexander Novikov, Matej Balog,	809
	M. Pawan Kumar, Emilien Dupont, Francisco J. R.	810
	Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar	811
	Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024.	
	Mathematical discoveries from program search with	
	large language models . <i>Nature</i> , 625:468–475.	
	Marta Sabou, Kalina Bontcheva, Leon Derczynski,	
	and Arno Scharl. 2014. Corpus annotation through	
	crowdsourcing: Towards best practice guidelines . In	
	<i>Proceedings of the 9th International Conference on</i>	
	<i>Language Resources and Evaluation (LREC)</i> , pages	
	859–866.	
	Shiori Sagawa, Pang Wei Koh, Tatsunori B. Hashimoto,	
	and Percy Liang. 2020. Distributionally robust neu-	
	ral networks for group shifts: On the importance of	
	regularization for worst-case generalization . In <i>In-</i>	
	<i>ternational Conference on Learning Representations</i>	
	<i>(ICLR)</i> . ArXiv:1911.08731.	
	Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane	
	Suhr. 2024. Quantifying language models’ sensitiv-	
	ity to spurious features in prompt design or: How i	
	learned to start worrying about prompt formatting .	
	In <i>International Conference on Learning Representa-</i>	
	<i>tions (ICLR)</i> . ArXiv:2310.11324.	
	Mikhail Seleznyov, Mikhail Chaichuk, Gleb Ershov,	
	Alexander Panchenko, Elena Tutubalina, and Oleg	
	Somov. 2025. When punctuation matters: A large-	
	scale comparison of prompt robustness methods for	
	LLMs . In <i>Findings of the Association for Computa-</i>	
	<i>tional Linguistics: EMNLP 2025</i> , pages 20370–	
	20385, Suzhou, China. Association for Computa-	
	tional Linguistics.	
	Burr Settles. 2009. Active learning literature survey .	
	Technical Report 1648, University of Wisconsin-	
	Madison.	
	Asankhaya Sharma. 2025. OpenEvolve: an open-source	
	evolutionary coding agent .	
	Rion Snow, Brendan O’Connor, Daniel Jurafsky, and	
	Andrew Y. Ng. 2008. Cheap and fast – but is it good?	
	Evaluating non-expert annotations for natural lan-	
	guage tasks . In <i>Proceedings of the 2008 Conference</i>	
	<i>on Empirical Methods in Natural Language Process-</i>	
	<i>ing (EMNLP)</i> , pages 254–263.	
	Zhen Tan, Dawei Li, Song Wang, Alimohammad	
	Beigi, Bohan Jiang, Amrita Bhattacharjee, Man-	
	sooreh Karami, Jundong Li, Lu Cheng, and Huan	
	Liu. 2024. Large language models for data annota-	
	tion and synthesis: A survey . In <i>Proceedings of the</i>	
	<i>2024 Conference on Empirical Methods in Natural</i>	
	<i>Language Processing (EMNLP)</i> . ArXiv:2402.13446.	
	Yu-Min Tseng, Wei-Lin Chen, Chung-Chi Chen, and	
	Hsin-Hsi Chen. 2025. Are expert-level language	
	models expert-level annotators? In <i>Conference on</i>	
	<i>Language Modeling (COLM)</i> . ArXiv:2410.03254.	

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. [Want to reduce labeling cost? GPT-3 can help](#). In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205. ArXiv:2108.13487.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. [Large language models as optimizers](#). In *International Conference on Learning Representations (ICLR)*. ArXiv:2309.03409.

Huaxiu Yao, Yu Wang, Sai Li, Linjun Zhang, Weixin Liang, James Zou, and Chelsea Finn. 2022. [Improving out-of-distribution robustness via selective augmentation](#). In *Proceedings of the 39th International Conference on Machine Learning (ICML)*. ArXiv:2201.00299.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. [Large language models are human-level prompt engineers](#). In *International Conference on Learning Representations (ICLR)*. ArXiv:2211.01910.

A Main Experiment Configuration

Table 2 summarizes the configuration of the main prompt-discovery run used to obtain the final PRIME prompt reported in Table 1. We omit implementation-only settings such as logging, checkpoint paths, retry counts, and the full mutator instruction text; the table retains the parameters that define the experimental protocol.

Component	Setting
Outer AL cycles	8
Evolution steps per cycle	15
Evolution implementation	OpenEvolve implementation of AlphaEvolve (Sharma, 2025)
Mutator model	gemin-2.5-pro, temperature 0.6
Worker ensemble	gpt-4o-mini; gemini-2.5-flash; claude-3.5-haiku
Worker decoding	Temperature 0.0; majority vote over 3 workers
Active batch	$B=80$, Hard ratio $\rho=0.7$
Hard threshold	$\tau=0.0$ for normalized worker disagreement
Embedding model	a11-MiniLM-L6-v2 for clustering and farthest-from-anchor sampling
Fitness	$0.5 \text{ Acc}_{\text{Hard}} + 0.3 \text{ Acc}_{\text{Anchor}} + 0.2 \kappa_{\text{Hard}} - \mathcal{P}_{\text{len}}$
Synthetic few-shot	Enabled; inject_as_hint; 8 examples; 4 boundary pairs
Pool expansion	Triggered below 15 Hard examples; refresh 10 examples per cycle
Evolution archive	Population 30; archive 500; 5 feature bins
Final prompt length	7148 tokens
Dataset	WILDS Amazon Reviews, all categories, user-disjoint splits

Table 2: Configuration of the main prompt-discovery run. The compact train/validation/development-test splits drive evolution and prompt selection; Table 1 reports the selected prompt on the full WILDS test set.

B API Usage and Cost Estimate

Table 3 summarizes token usage for the main prompt-discovery run. The reported logs account for ensemble evaluation and consolidation calls; mutator-token accounting was not available in the same log, so we estimate it as 8 cycles $\times$ 15 evolution steps, with approximately 30,000 input and 3,000 output tokens per mutation call.

Costs assume no prompt caching, batch discount, or free-tier usage, using per-million-token prices of gemini-2.5-flash (\$0.30 input, \$2.50 output), gpt-4o-mini (\$0.15 input, \$0.60 output), claude-3.5-haiku (\$0.80 input, \$4.00 output), and gemini-2.5-pro (\$1.25 input, \$10.00 output).

Call source	Input tokens	Output tokens	Cost
gemin-2.5-flash workers	12.15M	0.03M	\$3.71
gpt-4o-mini workers	11.51M	0.04M	\$1.75
claude-3.5-haiku workers	13.15M	0.51M	\$12.54
Consolidation, gemini-2.5-pro	0.07M	0.05M	\$0.58
Mutator estimate, gemini-2.5-pro	3.60M	0.36M	\$8.10
Total (reported + estimated)	40.48M	0.98M	\$26.67

Table 3: Approximate API cost of the main run. Reported calls account for 37.50M tokens and approximately \$18.57; adding the mutator estimate yields approximately 41.46M tokens and \$26.67.

C Fine-Tuned Baseline Details

Table 4 summarizes the public fine-tuned baselines used in Table 1. We do not rerun these baselines; we report the published Amazon-WILDS results from WILDS and LISA under the standard user-disjoint evaluation protocol.

D Dataset Category Distribution

Table 5 shows the distribution of reviews across product categories for the prompt-discovery splits summarized in Table 2. The training pool is sampled via clustering to ensure diverse coverage, while the validation and development test splits are constructed representatively across categories. The final prompt selected from this process is then evaluated on the full WILDS test set in Table 1.

E Synthetic Few-Shot Ablation

In the main WILDS experiments (Table 1), synthetic few-shot generation is enabled. To isolate its impact, we compare the full PRIME pipeline against an otherwise identical variant without synthetic few-shot generation. The generator targets adjacent-class confusions by creating minimal review pairs that differ in one rating-relevant cue. Injecting these boundary examples into `<FewShotExamples>` improves Hard-pool accuracy from 26.8% to 38.0%, indicating better calibration around difficult rating boundaries.

F Synthetic Few-Shot Example Set

Figure 4 shows an illustrative set of synthetic few-shot examples generated for the Amazon-Reviews

Baseline	Critical training hyperparameters (source)
ERM	DistilBERT-base-uncased; cross-entropy ERM; batch size 8; learning rate 1×10^{-5} with AdamW; L2 regularization 0.01; max sequence length 512; 3 epochs with early stopping on OOD validation; learning rate selected from $\{1 \times 10^{-6}, 2 \times 10^{-6}, 1 \times 10^{-5}, 2 \times 10^{-5}\}$ (WILDS (Koh et al., 2021))
GroupDRO	Same DistilBERT-base-uncased backbone and ERM hyperparameters; group-robust objective over reviewer/domain groups; GroupDRO step size fixed to 0.01 (WILDS (Koh et al., 2021; Sagawa et al., 2020))
LISA	DistilBERT backbone; selective manifold mixup for Amazon; intra-label strategy only ($p_{\text{sel}}=1.0$); $\lambda \sim \text{Beta}(2, 2)$; learning rate 2×10^{-6} ; batch size 8; Adam optimizer; weight decay 0; maximum 3 epochs (LISA (Yao et al., 2022))

Table 4: Critical configuration details for fine-tuned baselines compared against PRIME. All baselines are trained on labeled WILDS data and evaluated with the 10th-percentile user accuracy protocol used in Table 1.

ID	Category Name	Train	Validation	Dev. Test
0	Books	196	110	97
1	Kindle_Store	24	26	21
2	Video_Games	1	0	1
4	Arts_Crafts_and_Sewing	1	0	2
5	Toys_and_Games	14	0	2
6	Automotive	4	0	2
7	Tools_and_Home_Improvement	16	1	7
8	Sports_and_Outdoors	4	3	3
10	Prime_Pantry	5	0	1
11	Pet_Supplies	9	12	7
12	CDs_and_Vinyl	10	7	15
13	Patio_Lawn_and_Garden	3	0	3
14	Office_Products	3	2	8
15	Cell_Phones_and_Accessories	2	0	2
16	Clothing_Shoes_and_Jewelry	12	5	11
18	Movies_and_TV	15	26	6
20	Electronics	13	8	7
21	Grocery_and_Gourmet_Food	19	10	11
23	Luxury_Beauty	0	0	1
24	Home_and_Kitchen	23	14	18
25	Industrial_and_Scientific	1	1	0

Table 5: Distribution of reviews across product categories in the prompt-discovery splits.

task. The generator produces only the review texts and target ratings; the highlights in the figure are added manually for visualization. The examples cover the full ordinal scale and illustrate the cues that should move a review across adjacent rating boundaries.

G Consolidation and Rule Promotion Detail

Figure 5 illustrates the interaction between active-learning cycles and consolidation in greater detail. The left side shows successive cycle-level rule edits, including additions and removals for rating-specific boundaries and edge-case handling. After each active-learning cycle, a consolidation step filters these local edits, promoting stable rules into <BaseGuidelines> while keeping more tentative, boundary-specific heuristics in <DynamicRules>.

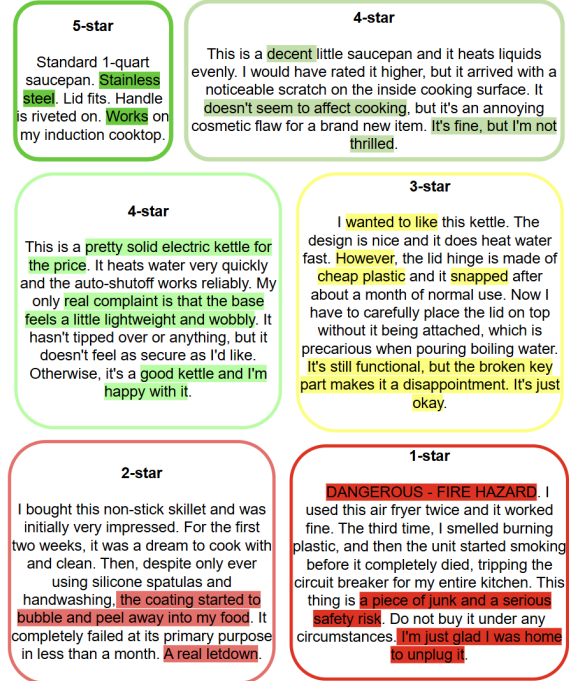


Figure 4: Synthetic few-shot examples for ordinal sentiment boundaries. The generator creates controlled review texts covering positive, mixed, negative, and severe-failure cases. Highlights are manual visual annotations, not generator output. The generated texts are injected into <FewShotExamples> to clarify adjacent class boundaries for the LLM ensemble.

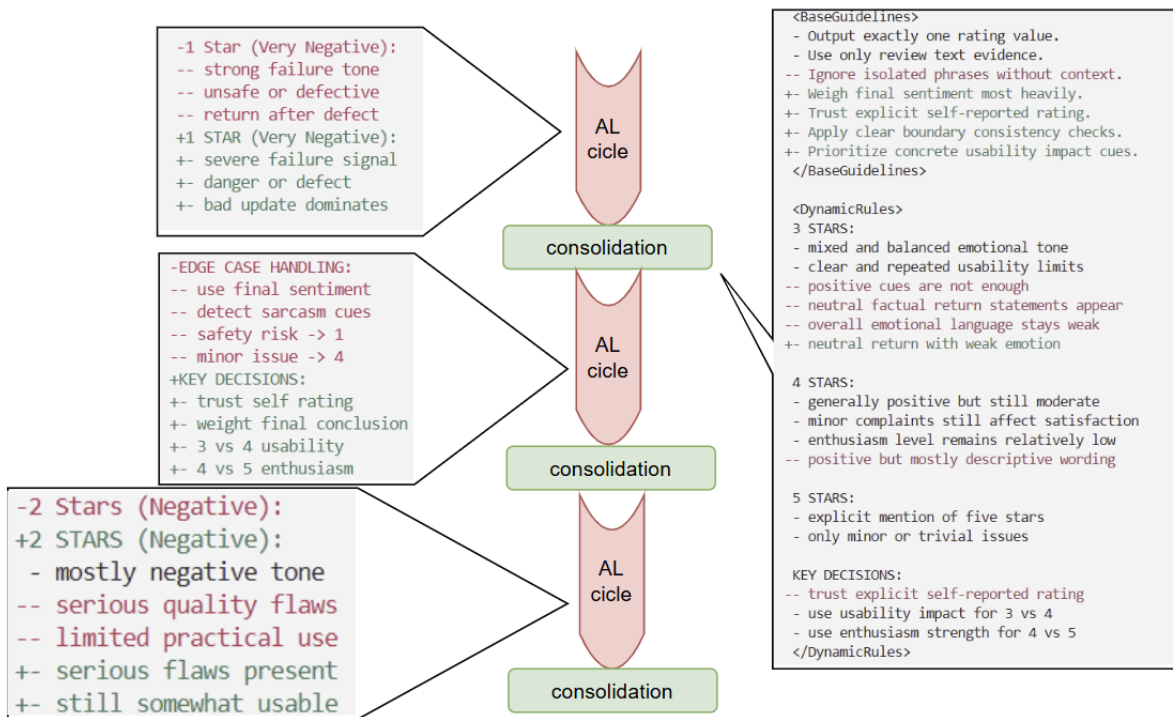


Figure 5: Detailed consolidation process across active-learning cycles. Cycle-level edits are repeatedly produced during active learning, then consolidated into the structured prompt. Stable, broadly useful rules are promoted into <BaseGuidelines>, while more local rating-boundary heuristics remain in <DynamicRules>.